SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: **Artificial Intelligence** **COURSE CODE:CSA1708**

COURSE OUTCOME COVERED

CO1:ApplyAIbasics suchasintelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5

Duration : 1Hour
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I Ganesh Ram V (1924260258) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question 1: Water Jug Problem

Problem formulation

State: (x, y) where x=gallons in the 4-gallon jug ($0 \le x \le 4$) and y = gallons in the 3-gallon jug ($0 \le y \le 3$).

Initial state: (0, 0)

Goal state: (2, n) –exactly 2 gallons in the 4-gallon jug (n can be anything).

Actions (production rules):

No.	Condition → Result	Meaning
1	(x, y), $x < 4 \rightarrow$ (4, y)	Fill the 4-gallon jug from the pump
2	(x, y), $y < 3$ (x, 3)	Fill the 3-gallon jug from the pump
3	(x, y), $x > 0 \rightarrow$ (0, y)	
4	(x, y), $y > 0 \rightarrow$ (x, 0)	Empty the 4-gallon jug on the ground
5	(x, y), $x+y \ge 4, y > 0 \rightarrow$ (4, y−(4−x))	Empty the 3-gallon jug on the ground
6	(x, y), $x+y \ge 3, x > 0 \rightarrow$ (x−(3−y), 3)	Pour from 3-gal into 4-gal until the 4-gal is full
7	(x, y), $x+y \le 4, y > 0 \rightarrow$ (x+y, 0)	Pour from 4-gal into 3-gal until the 3-gal is full
8	(x, y), $x+y \le 3, x > 0 \rightarrow$ (0, x+y)	Pour all of the 3-gal into the 4-gal
		Pour all of the 4-gal into the 3-gal

Solution (found by breadth-first search –shortest, 6 steps)

Step	4-gal jug (x)	3-gal jug (y)	Action / Rule applied
0	0	0	Initial state
1	0	3	Fill the 3-gallon jug (Rule 2)
2	3	0	Pour all of the 3-gal into the 4-gal (Rule 7)
3	3	3	Fill the 3-gallon jug again (Rule 2)
4	4	2	Pour from the 3-gal into the 4-gal until it is full (Rule 5); 1 gallon goes in, 2 remain
5	0	2	Empty the 4-gallon jug (Rule 3)
6	2	0	Pourall of the 3-gal into the 4-gal (Rule 7) → GOAL

Alternative solution (also 6 steps): (0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3). The last state has 2 gallons in the4-gallon jug.

Result: Exactly 2 gallons is obtained in the 4-gallon jug; the state path is (0,0) → (0,3) → (3,0) →(3,3) →(4,2) →(0,2) →(2,0).

Question 2: Mars Rover –PEAS Analysis

AMarsrover isanautonomous agentthat travelson theMartian surface, and collects, analyses and explores soil/rock samples. Its design is analysed using the PEAS framework.

(a) Percepts (what the sensors give the agent)

- Camera images (stereo/panoramic, hazard-avoidance cameras) –terrain, rocks,
- slopes, obstacles, interesting samples.
- Spectrometer / chemical analysis readings (e.g. composition of rocks and soil,
- presence of water or minerals).
- Position and orientation: GPS-like navigation, wheel odometry, gyroscope,
- accelerometer, tiltsensors.
- Wheel/motor current and wheel slip –detects getting stuck in sand.
  Internal status: battery level, solar-panel power, temperature, fuel, system health.
  Environmental data: wind, dust, ambient temperature, radiation.
  Commands and messages from Earth control (with communication delay).

(b) Operating environment

Property	Characterisation
Ob s erv abi lity	Partially observable –sensors have limited range; terrain beyond view is unknown.
A gents	Single agent(therover) –other agents(Earthcontrol)only send c om mand s.
D eterm inis m	Stochastic–wheelslip, duststorms,unpredictableterrain and sensor noise.
Episodic / Sequential	Sequential–acurrentmovementaffectsfutureposition,power and safety.
Static / Dynamic	Dynamic –weather, light, temperature and dust change while the rover is deciding.
Discrete / Continuous	Continuous –positions, speeds, time and sensor values are con tinuo us.
Known / Unknown	Largely unknown –unexplored terrain; the rover must learn and mapit.

(c) Actions (actuators)

- Move forward / backward, turn left / right, change speed, stop, climb or avoid obstacles (wheels/motors, steering).
- Deploy andmovetheroboticarm; drill,scooporgrindrockand soil; collectandstore samples.
- Point cameras and instruments; take images; run spectrometer / on-board laboratory analysis.
- Orient solar panels, switch to sleep/power-saving mode, heat or cool components.
- Transmit data to Earth / orbiter and receive commands (antenna).

(d) Performance measure

- Number and scientific value of samples collected and analysed (quality and variety).
- Area of terrain explored and mapped; distance travelled towards target sites.
- Safety: number of collisions, falls, or times the rover got stuck (should be minimal).
- Energy efficiency: power used per task; the rover stays within its power budget.
- Mission lifetime and reliability (operating days without failure).
- Quantity and quality of data successfully transmitted to Earth; time taken per task.

(e) Most suitable agent architecture

A goal-based agent with utility-based decision making and learning (a hybrid / layered architecture) isthemostsuitable.Reasons:

- **Goals:** the rover has explicit goals (reach a site, analyse a rock) and must plan a sequence of actions –so a simple reflex agent is not enough.
- **Partial observability:** it must maintain an internal model/map of the world and its own state, so a model-based agent is needed.
- **Trade-offs:** there are competing aims (science value vs. battery vs. risk), so a utility functionisneeded tochoosethebest action.
- **Communication delay:** Earth cannot control it in real time, so the rover must act autonomously.
- **Learning:**the environmentis unknown,solearningimproves terrainclassification and navigation over time.
- **Reflex layer:** a low-level reflex layer is added for immediate safety (stop if a cliff or tilt is detected).

Question 3: 8-Queens Problem Formulation

Place 8 queens on an 8×8 chessboard so that no queen attacks another (no two share a row, column or diagonal). In the figure given, the queen in the first column and the queen in the last column lie on the same diagonal, so it is not a solution (it has at least one attacking pair).

Problem components (incremental formulation)

Component	Definition
States	Any arrangement of 0 to8queensontheboardsuchthatnoqueen attacks another (queens are added column by column, leftmost first).
Initial state	The empty board (no queens placed).
Ac tio ns	Add a queen to a square in the leftmost empty column that is not attacked by any queen already on the board.
Transition model	Returns the board with the new queen added in the chosen square.
Goal test	8 queens are on the board and none is attacked.
Path cost	Notimportant –only the final configuration matters (each step has cost 1 if need ed) .

Search space comparison

Formulation	State space size	Remark
Complete-state: all 8 queens placed, any squares, move q ueens	≈ 64×63×…×57 ≈ 1.8 × 10^{14} sequences (C(64,8) ≈ 4.4 ×10^{9} s tate s)	Verylarge;uninformed search is imp ra c tica l
Incremental, onequeen per column, any row	8^{8} ≈ 1.67 ×10^{7}	Smaller, butstillcontains attacking s tate s
Incremental, add only non-attacked queens (chosen)	Only 2,057 states	Very small; easily solved by search

Exploring the search space with uninformed strategies

- **Depth-first search with backtracking:** place a queen in column 1, then in column 2 on a safe square, and so on. When a column has no safe square, backtrack to the previous column and try the next row. Memory needed is only O(b·m) and this is the usual method for 8-queens.

- **Breadth-first search:** generates all safe partial placements level by level (level k = k queens). It finds all solutions but needs a lot of memory as the levels grow.
- **Iterative deepening / depth-limited search (limit = 8):** all solutions lie at depth exactly 8, so DFS with limit 8 is complete here without the memory cost of BFS.

Example solution (found by DFS with backtracking)

Rowpositionsforcolumns1to8:(1,5,8,6,3,7,2,4).The8-queens problem has 92 distinct solutions (12 unique up to rotation and reflection).

```
Q . . . . . . .
. . . .
. . . . . Q . . Q .
. . .           .
. Q . . . . . . . Q
. . .           .
. . . Q . . . . .
. . Q . . . Q . . .
```

Check: all rows are different, one queen per column, and no two queens have the same value of (row − column) or (row + column), so there are no diagonal attacks.

Question 4: OLA Cab Booking – Problem-Solving Agent

Type of agent

A goal-based, problem-solving agent (with utility-based selection of the cab type) is suitable. The goal is clear (reach the destination from the source). The agent must look aheadand search over possibleroutesand cabchoices, and selectthe option that best satisfies the customer's preferences: cost, comfort, time and availability. The environment is observable (map, traffic, faresandavailabilityshownin the app),single-agent (the customer), mostly deterministic, sequential and dynamic (traffic, fares).

Problem formulation

Component	Definition
Initial state	Customer at pickup location with no ride booked.
Goal state	Customer reaches the destination location using the selected cab.
Ac tio ns	Enter pickup/drop; view cab types (Mini, Micro, Sedan, Shared, Prime…); select cab; confirm booking; track the driver; board and travel; pay.
Transition model	Each action moves the state: Searching →Cab selected →Booking confirmed →Driver assigned →Trip in progress →Destination reached.
Path cost / utility	Fare + waiting time + travel time (+ comfort preference). Lower cost and higher comfort givehigher utility.

Pseudocode

```
function OLA-BOOKING-AGENT(percept) returns an action
  persistent: state, goal, problem, plan (sequence of actions), preferences

  state <-UPDATE-STATE(state, percept)     // location, traffic, fares
  if plan is empty then
      goal <-FORMULATE-GOAL(state)        // reach D from S
      problem <-FORMULATE-PROBLEM(state, goal) // states, actions, cost
      plan <-SEARCH(problem)              // best route + cab
      if plan = failure then return NO-RIDE-AVAILABLE
  action <-FIRST(plan)
  plan <-REST(plan)
  return action

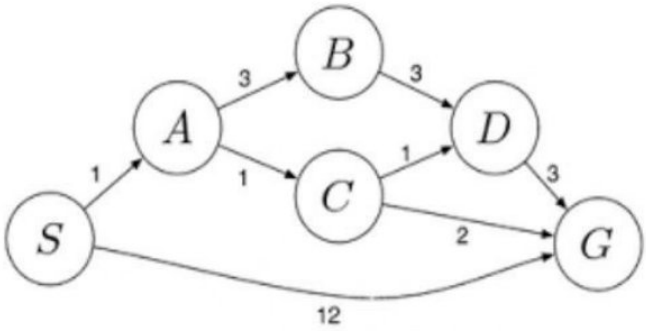
function SEARCH(problem) returns plan
  available <-GET-AVAILABLE-CABS(source)    // Mini, Micro, Sedan...
  best <-NULL; bestUtility <--infinity
```

```
for each cab in available do
    route <-FIND-SHORTEST-ROUTE(source, destination) // UCS or A* on map
    fare <-ESTIMATE-FARE(cab, route)
    eta   <-ESTIMATE-TIME(cab, route)
    utility <-w1*comfort(cab) -w2*fare -w3*eta          // preference weights
    if utility > bestUtility then best <-(cab, route); bestUtility <-utility
if best = NULL then return failure
return [CONFIRM-BOOKING(best.cab), ASSIGN-DRIVER, TRACK-DRIVER,
        BOARD, TRAVEL(best.route), REACH-DESTINATION, PAY]
```

The agent then executes the plan step by step; if a percept changes (cab cancelled, traffic jam, fare change), the plan is empty or invalid and the agent re-plans.

Question 5: Uniform Cost Search –Warehouse Network

Graph (from the given figure)



Edges and costs: S→A = 1, S→G = 12, A→B = 3, A→C = 1, B→D = 3, C→D = 1, C→G = 2, D→G = 3.

Method

UCS always expands the node in the frontier (priority queue) with the lowest path cost g(n). Thegoaltestisappliedwhen anodeisselectedforexpansion,notwhenitisgenerated,

which guarantees an optimal path.

Step-by-step execution

Step	Node expanded (g)	Nodes added / updated	Frontier after step (node:cost)
1	S (0)	A = 0+1 = 1; G = 0+12 = 12	A:1, G:12
2	A (1)	B = 1+3 = 4; C = 1+1 = 2	C:2, B:4, G:12
3	C (2)	D = 2+1 = 3; G = 2+2 = 4 (better than 12, so update) G = 3+3 = 6 (worse than 4,	D:3, B:4, G:4
4	D (3)	ig nor e)	B:4, G:4
5	B (4)	D= 4+3 = 7 (worse than 3, ig nor e)	G:4
6	G (4)	Goal selected for expansion → STOP	–

Explored (closed) list in order: S, A, C, D, B, G.

Result

Least-cost path: S →A →C →G

Total minimum cost: 1 + 1 + 2 = 4 units (fuel + time).

Interpretation: The direct route S → G costs 12, and the other route S → A → C → D → G costs 6. The optimal route through warehouses A and C costs only 4, saving 8 units compared to the direct route and 2 units compared to the next best route. UCS is complete andoptimalherebecause alledgecosts are positive. Itstimeand space complexityis O(b^(1+⌊C*/ε⌋)),whereC* is theoptimal cost and ε the minimum edge cost.